

Sorting and Searching

```
#include <iostream>
#include <vector>
#include <algorithm>

using namespace std;

// Utility to print array
void printArray(std::vector<int>& arr) {
    for (int x : arr) std::cout << x << " ";
    std::cout << "\n";
}

// Bubble Sort
void bubbleSort(vector<int>& arr) {
    int n = arr.size();
    for (int i = 0; i < n-1; ++i){
        cout << "Swapping " ;
        for (int j = 0; j < n-i-1; ++j)
            if (arr[j] > arr[j+1]){
                cout << "" << arr[j] << " and " << arr[j+1] << ", ";
                swap(arr[j], arr[j+1]);
            }
        cout << "\n";
    }
}

void bubbleSortstop(vector<int>& arr) {
    int n = arr.size();
    bool swapped;
    for (int i = 0; i < n-1; ++i) {
        swapped = false;
        for (int j = 0; j < n-i-1; ++j) {
            if (arr[j] > arr[j+1]) {
                swap(arr[j], arr[j+1]);
                swapped = true;
            }
        }
        if (!swapped) break;
    }
}
```

```

// Insertion Sort
void insertionSort(vector<int>& arr) {
    int n = arr.size();
    for (int i = 1; i < n; ++i) {
        int key = arr[i], j = i - 1;

        while (j >= 0 && arr[j] > key) {
            arr[j+1] = arr[j]; j--;
        }
        arr[j+1] = key;
    }
}

// Shell Sort
void shellSort(vector<int>& arr) {
    int n = arr.size();
    for (int gap = n/2; gap > 0; gap /= 2) {
        cout << "Gap: " << gap << "\n";
        printArray(arr);
        for (int i = gap; i < n; ++i) {
            int temp = arr[i], j = i;
            while (j >= gap && arr[j-gap] > temp) {
                arr[j] = arr[j-gap];
                cout << "Array during moving " << arr[j-gap] << " to position " << j
<< ": ";
                printArray(arr);
                j -= gap;
            }
            arr[j] = temp;
            // cout << "\n";
            cout << "Array after current gap moving " << temp << " gap " << gap << ":
";
            printArray(arr);
        }
        cout << "\n";
        cout << "Array after gap " << gap << ": ";
        printArray(arr);
    }
}

// Selection Sort
void selectionSort(vector<int>& arr) {
    int n = arr.size();
    for (int i = 0; i < n-1; ++i) {
        int minIdx = i;
        for (int j = i+1; j < n; ++j)
            if (arr[j] < arr[minIdx])

```

```

        minIdx = j;
        std::swap(arr[i], arr[minIdx]);
    }
}

// Merge Sort Helper
void merge(vector<int>& arr, int l, int m, int r) {
    int n1 = m-l+1, n2 = r-m;
    vector<int> L(n1), R(n2);
    cout << "\nMerging parts:\n";
    cout << "Left part: ";
    cout << "[ ";
    for (int i = l; i <= m; ++i) cout << arr[i] << " ";
    cout << "]\n";
    cout << "Right part: ";
    cout << "[ ";
    for (int i = m+1; i <= r; ++i) cout << arr[i] << " ";
    cout << "]\n";
    for (int i = 0; i < n1; ++i) L[i] = arr[l+i];
    for (int i = 0; i < n2; ++i) R[i] = arr[m+1+i];
    int i = 0, j = 0, k = l;
    cout << "Merging: ";
    while (i < n1 && j < n2)
        arr[k++] = (L[i] <= R[j]) ? L[i++] : R[j++];
    cout << "\n";
    cout << "After merging left and right parts: ";
    cout << "[ ";
    for (int w = l; w < k; w++)
    {
        cout << arr[w] << " ";
    }
    cout << "]\n";

    while (i < n1) arr[k++] = L[i++];
    while (j < n2) arr[k++] = R[j++];
    cout << "Merged the rest of another array (L or R): ";

    cout << "[ ";
    for (int p = l; p <= r; ++p) cout << arr[p] << " ";
    cout << "]" Done. let go up ^^ \n";
}

// Merge Sort
void mergeSort(std::vector<int>& arr, int l, int r) {
    cout << "Current array segment: ";
    cout << "[ ";
    for (int i = l; i <= r; ++i) cout << arr[i] << " ";
    cout << "]\n";

```

```

    if (l < r) {
        int m = l + (r-l)/2;
        cout << "Dividing: ";

        cout << "[ ";
        for (int i = l; i <= m; ++i) cout << arr[i] << " ";
        cout << "] -- ";
        cout << "[ ";
        for (int i = m+1; i <= r; ++i) cout << arr[i] << " ";
        cout << "]\n";
        cout << "\nLeft part: ";
        mergeSort(arr, l, m);

        cout << "\nRight part: ";

        mergeSort(arr, m+1, r);

        merge(arr, l, m, r);
    }
}

int binarySearch(const std::vector<int>& arr, int target) {
    int left = 0, right = arr.size() - 1;
    while (left <= right) {
        int mid = left + (right - left) / 2;
        if (arr[mid] == target) return mid;
        else if (arr[mid] < target) left = mid + 1;
        else right = mid - 1;
    }
    return -1;
}

int binarySearchRecursive(const std::vector<int>& arr, int left, int right, int target) {
    if (left > right) return -1;
    int mid = left + (right - left) / 2;
    if (arr[mid] == target)
        return mid;
    else if (arr[mid] < target)
        return binarySearchRecursive(arr, mid + 1, right, target);
    else
        return binarySearchRecursive(arr, left, mid - 1, target);
}

int interpolationSearch(const std::vector<int>& arr, int target) {
    int low = 0, high = arr.size() - 1;
    while (low <= high && target >= arr[low] && target <= arr[high]) {

```

```

        if (low == high) {
            if (arr[low] == target) return low;
            return -1;
        }
        int pos = low + ((double)(high - low) / (arr[high] - arr[low])) * (target -
arr[low]);
        if (arr[pos] == target) return pos;
        if (arr[pos] < target) low = pos + 1;
        else high = pos - 1;
    }
    return -1;
}

int interpolationSearchRecursive(const std::vector<int>& arr, int low, int high, int
target) {
    if (low <= high && target >= arr[low] && target <= arr[high]) {
        if (low == high) {
            if (arr[low] == target) return low;
            return -1;
        }
        int pos = low + ((double)(high - low) / (arr[high] - arr[low])) * (target -
arr[low]);
        if (arr[pos] == target) return pos;
        if (arr[pos] < target) return interpolationSearchRecursive(arr, pos + 1, high,
target);
        else return interpolationSearchRecursive(arr, low, pos - 1, target);
    }
    return -1;
}

int findMin(const std::vector<int>& arr) {
    int minVal = arr[0];
    for (int x : arr)
        if (x < minVal) minVal = x;
    return minVal;
}

int findMinRecursive(const std::vector<int>& arr, int n) {
    if (n == 1) return arr[0];
    int minRest = findMinRecursive(arr, n - 1);
    return arr[n - 1] < minRest ? arr[n - 1] : minRest;
}

int findMax(const std::vector<int>& arr) {
    int maxVal = arr[0];
    for (int x : arr)
        if (x > maxVal) maxVal = x;
    return maxVal;
}

```

```

}

int findMaxRecursive(const std::vector<int>& arr, int n) {
    if (n == 1) return arr[0];
    int maxRest = findMaxRecursive(arr, n - 1);
    return arr[n - 1] > maxRest ? arr[n - 1] : maxRest;
}

int findMinDivide(const vector<int>& arr, int left, int right) {
    if (left == right) return arr[left];
    int mid = left + (right - left) / 2;
    int minLeft = findMinDivide(arr, left, mid);
    int minRight = findMinDivide(arr, mid + 1, right);
    return min(minLeft, minRight);
}

int findMaxDivide(const vector<int>& arr, int left, int right) {
    if (left == right) return arr[left];
    int mid = left + (right - left) / 2;
    int maxLeft = findMaxDivide(arr, left, mid);
    int maxRight = findMaxDivide(arr, mid + 1, right);
    return max(maxLeft, maxRight);
}

int partition_basic(std::vector<int>& arr, int low, int high) {
    int pivot = arr[low];
    int i = low - 1;
    int j = high + 1;
    while (true) {
        do { ++i; } while (arr[i] < pivot);
        do { --j; } while (arr[j] > pivot);
        if (i >= j)
            break;
        std::swap(arr[i], arr[j]);
    }

    std::swap(arr[low], arr[j]);
    return j;
}

int partition(std::vector<int>& arr, int low, int high) {
    int pivot = arr[high], i = low-1;
    for (int j = low; j < high; ++j) {

```

```

        if (arr[j] < pivot)
            std::swap(arr[++i], arr[j]);
    }
    std::swap(arr[i+1], arr[high]);
    return i+1;
}

// Quick Sort
void quickSort(std::vector<int>& arr, int low, int high) {
    if (low < high) {
        int pi = partition_basic(arr, low, high);
        cout << "Pivot index: " << pi << ", Pivot value: " << arr[pi] << "\n";
        cout << "Array state: ";
        printArray(arr);
        quickSort(arr, low, pi-1);
        quickSort(arr, pi+1, high);
    }
}

int medianOfThree(std::vector<int>& arr, int low, int high) {
    int mid = low + (high - low) / 2;
    if (arr[low] > arr[mid]) std::swap(arr[low], arr[mid]);
    if (arr[low] > arr[high]) std::swap(arr[low], arr[high]);
    if (arr[mid] > arr[high]) std::swap(arr[mid], arr[high]);
    // arr[mid] is median, move it to high-1 for partitioning
    std::swap(arr[mid], arr[high - 1]);
    return arr[high - 1];
}

int partitionMedian(vector<int>& arr, int low, int high) {
    int pivot = medianOfThree(arr, low, high);
    int i = low, j = high - 1;
    while (true) {
        while (arr[++i] < pivot);
        while (arr[--j] > pivot);
        if (i < j)
            swap(arr[i], arr[j]);
        else
            break;
    }
    std::swap(arr[i], arr[high - 1]);
    return i;
}

void quickSortMedian(std::vector<int>& arr, int low, int high) {
    if (low + 2 <= high) {

```

```

        int pi = partitionMedian(arr, low, high);
        cout << "Pivot index: " << pi << ", Pivot value: " << arr[pi] << "\n";
        cout << "Array state: "; printArray(arr);
        quickSortMedian(arr, low, pi - 1);
        quickSortMedian(arr, pi + 1, high);
    } else if (low < high) {
        if (arr[low] > arr[high])
            std::swap(arr[low], arr[high]);
    }
}

int main() {
    std::vector<int> arr = {5, 2, 9, 1, 5, 6};

    std::vector<int> arr1 = arr;
    bubbleSort(arr1);
    std::cout << "Bubble Sort: "; printArray(arr1);

    std::vector<int> arr2 = arr;
    insertionSort(arr2);
    std::cout << "Insertion Sort: "; printArray(arr2);

    std::vector<int> arr3 = arr;
    cout << "Shell Sort Steps:\n";
    shellSort(arr3);
    std::cout << "Shell Sort: "; printArray(arr3);

    std::vector<int> arr4 = arr;
    selectionSort(arr4);
    std::cout << "Selection Sort: "; printArray(arr4);

    std::vector<int> arr5 = arr;
    mergeSort(arr5, 0, arr5.size()-1);
    std::cout << "Merge Sort: "; printArray(arr5);

    std::vector<int> arr6 = arr;
    cout << "Quick Sort Steps:\n";
    quickSort(arr6, 0, arr6.size()-1);
    std::cout << "Quick Sort: "; printArray(arr6);

    return 0;
}

```


DAC

```
#include <iostream>
#include <vector>
#include <iomanip>
#include <cmath>

using namespace std;
long long intPow10(int exp) {
    long long res = 1;
    while (exp--) res *= 10;
    return res;
}

long long karatsuba(long long x, long long y, int depth = 0) {
    string indent(depth * 2, ' ');
    cout << indent << "Karatsuba called with x = " << x << ", y = " << y << endl;

    // Base case
    if (x < 10 || y < 10) {
        cout << indent << "Base case: " << x << " * " << y << " = " << x * y << endl;
        return x * y;
    }

    int len_x = to_string(x).size();
    int len_y = to_string(y).size();
    int n = max(len_x, len_y);
    if (n % 2 != 0) ++n;

    int m = n / 2;
    long long pow10m = intPow10(m);

    // แบ่งจำนวน
    long long high1 = x / pow10m;
    long long low1 = x % pow10m;
    long long high2 = y / pow10m;
    long long low2 = y % pow10m;

    cout << indent << "Split x: " << x << " into " << high1 << " and " << low1 <<
endl;
    cout << indent << "Split y: " << y << " into " << high2 << " and " << low2 <<
endl;

    cout << indent << "a: " << high1 << ", b: " << low1 << endl;
```

```

    cout << indent << "c: " << high2 << ", d: " << low2 << endl;
    cout << indent << "(a*c): " << high1 << " * " << high2 << endl;
    long long z2 = karatsuba(high1, high2, depth + 1);
    cout << indent << "b*d: " << low1 << " * " << low2 << endl;
    long long z0 = karatsuba(low1, low2, depth + 1);
    cout << indent << "(a+b)*(c+d): " << "(" << high1 << "+" << low1 << ")" << " * " <<
    "(" << high2 << "+" << low2 << ")" << endl;
    long long z1 = karatsuba(low1 + high1, low2 + high2, depth + 1);

    long long result = z2 * intPow10(2 * m) + (z1 - z2 - z0) * pow10m + z0;
    cout << indent << "Result: " << result << endl;
    return result;
}

void printMatrix(const vector<vector<long long>>& mat, string name = "", int depth =
0) {
    string indent(depth * 2, ' ');
    cout << indent;
    if (!name.empty()) cout << name << ":\n";
    for (const auto& row : mat) {
        for (auto v : row) cout << indent << setw(6) << v;
        cout << "\n";
    }
    cout << endl;
}

// เช็ท กำลัง 2 ไหม
int nextPow2(int x) {
    int p = 1;
    while (p < x) p <= 1;
    return p;
}

// ปรับขนาด matrix
vector<vector<long long>> resizeMatrix(
    const vector<vector<long long>>& mat,
    int newR, int newC
) {
    int oldR = mat.size();
    int oldC = mat[0].size();

    vector<vector<long long>> resized(newR, vector<long long>(newC, 0));

    for (int i = 0; i < min(oldR, newR); ++i)
        for (int j = 0; j < min(oldC, newC); ++j)
            resized[i][j] = mat[i][j];

    return resized;
}

```

```

// บวกสอง matrix เอาไว้ใช้งาน
vector<vector<long long>> add(
    const vector<vector<long long>>& A,
    const vector<vector<long long>>& B
) {
    int n = A.size();
    vector<vector<long long>> C(n, vector<long long>(n));
    for (int i = 0; i < n; ++i)
        for (int j = 0; j < n; ++j)
            C[i][j] = A[i][j] + B[i][j];
    return C;
}

vector<vector<long long>> sub(
    const vector<vector<long long>>& A,
    const vector<vector<long long>>& B
) {
    int n = A.size();
    vector<vector<long long>> C(n, vector<long long>(n));
    for (int i = 0; i < n; ++i)
        for (int j = 0; j < n; ++j)
            C[i][j] = A[i][j] - B[i][j];
    return C;
}

// Split matrix ตามขนาดครึ่งหนึ่ง
void split(
    const vector<vector<long long>>& A,
    vector<vector<long long>>& A11,
    vector<vector<long long>>& A12,
    vector<vector<long long>>& A21,
    vector<vector<long long>>& A22
) {
    int k = A.size() / 2;
    for (int i = 0; i < k; ++i)
        for (int j = 0; j < k; ++j) {
            A11[i][j] = A[i][j];
            A12[i][j] = A[i][j + k];
            A21[i][j] = A[i + k][j];
            A22[i][j] = A[i + k][j + k];
        }
}

// Strassen (square matrices)

```

```

vector<vector<long long>> strassen(
    const vector<vector<long long>>& A,
    const vector<vector<long long>>& B,
    int depth = 0
) {
    int n = A.size();

    if (n == 1) {
        cout << string(depth * 2, ' ') << "Base case reached at depth " << depth <<
endl;
        return {{A[0][0] * B[0][0]}};
    }

    int k = n / 2;
    vector<vector<long long>> A11(k, vector<long long>(k)),
        A12(k, vector<long long>(k)),
        A21(k, vector<long long>(k)),
        A22(k, vector<long long>(k));

    vector<vector<long long>> B11(k, vector<long long>(k)),
        B12(k, vector<long long>(k)),
        B21(k, vector<long long>(k)),
        B22(k, vector<long long>(k));

    split(A, A11, A12, A21, A22);
    printMatrix(A11, "Matrix A11", depth);
    printMatrix(A12, "Matrix A12", depth);
    printMatrix(A21, "Matrix A21", depth);
    printMatrix(A22, "Matrix A22", depth);
    split(B, B11, B12, B21, B22);
    printMatrix(B11, "Matrix B11", depth);
    printMatrix(B12, "Matrix B12", depth);
    printMatrix(B21, "Matrix B21", depth);
    printMatrix(B22, "Matrix B22", depth);

    cout << string(depth * 2, ' ') << "Strassen called at depth " << depth << " with
matrix size " << n << "x" << n << endl;
    auto M1 = strassen(add(A11, A22), add(B11, B22), depth + 1);
    auto M2 = strassen(add(A21, A22), B11, depth + 1);
    auto M3 = strassen(A11, sub(B12, B22), depth + 1);
    auto M4 = strassen(A22, sub(B21, B11), depth + 1);
    auto M5 = strassen(add(A11, A12), B22, depth + 1);
    auto M6 = strassen(sub(A21, A11), add(B11, B12), depth + 1);
    auto M7 = strassen(sub(A12, A22), add(B21, B22), depth + 1);

    auto C11 = add(sub(add(M1, M4), M5), M7);
    auto C12 = add(M3, M5);

```

```

    auto C21 = add(M2, M4);
    auto C22 = add(sub(add(M1, M3), M2), M6);

    vector<vector<long long>> C(n, vector<long long>(n));
    for (int i = 0; i < k; ++i)
        for (int j = 0; j < k; ++j) {
            C[i][j] = C11[i][j];
            C[i][j + k] = C12[i][j];
            C[i + k][j] = C21[i][j];
            C[i + k][j + k] = C22[i][j];
        }
    printMatrix(C, "Result C at depth " + to_string(depth), depth);

    return C;
}

vector<vector<long long>> multiplyStrassen(
    const vector<vector<long long>>& A,
    const vector<vector<long long>>& B
) {
    int rA = A.size();
    int cA = A[0].size();
    int rB = B.size();
    int cB = B[0].size();

    if (cA != rB) {
        throw runtime_error("Invalid multiplication dimensions");
    }

    int N = nextPow2(max({rA, cA, rB, cB}));

    auto A_pad = resizeMatrix(A, N, N);
    auto B_pad = resizeMatrix(B, N, N);
    printMatrix(A_pad, "Matrix A_pad");
    printMatrix(B_pad, "Matrix B_pad");

    auto C_pad = strassen(A_pad, B_pad);

    return resizeMatrix(C_pad, rA, cB);
}

struct Point { long long x, y; };

```

```

static bool lexLess(const Point& a, const Point& b) {
    if (a.x != b.x) return a.x < b.x;
    return a.y < b.y;
}

vector<Point> maximaSet(vector<Point> S) {
    if (S.size() <= 1) return S;

    sort(S.begin(), S.end(), lexLess);
    cout << "-----\nSorted points: ";
    for (int i = 0; i < S.size(); i++)
    {
        cout << "(" << S[i].x << "," << S[i].y << "), ";
    }
    cout << endl;

    // เรียงแล้วหาจุดกึ่งกลาง
    Point p = S[S.size() / 2];
    cout << "Median point p: (" << p.x << "," << p.y << ")\n";

    // แยกซ้าย L กับ G
    vector<Point> L, G;
    for (auto &pt : S) {
        if (lexLess(pt, p)) L.push_back(pt);
        else G.push_back(pt);
    }
    cout << "L points: ";
    for (auto &pt : L) {
        cout << "(" << pt.x << "," << pt.y << "), ";
    }
    cout << endl;
    cout << "G points: ";
    for (auto &pt : G) {
        cout << "(" << pt.x << "," << pt.y << "), ";
    }
    cout << endl;
    cout << "Call L\n";
    // เรียกใช้ recursive กับ L กับ G
    auto M1 = maximaSet(L);
    cout << "Call G\n";
    auto M2 = maximaSet(G);

    // เอาค่าน้อยสุดมาเลย ไปตัดฝั่ง M1
    Point q = *min_element(M2.begin(), M2.end(), lexLess);

    // ลบจุดที่โดนครอบใน M1 จะได้เหลือน้อยๆ
    vector<Point> res;

```

```

    for (auto &r : M1) {
        if (!(r.x <= q.x && r.y <= q.y)) res.push_back(r);
    }
    cout << "After removing dominated points in M1 using q: (" << q.x << ", " << q.y <<
")\n";
    cout << "Remaining points in M1:\n";
    for (auto &pt : res) {
        cout << "" << pt.x << ", " << pt.y << "), ";
    }
    cout << endl;
    cout << "Points in M2:\n";
    for (auto &pt : M2) {
        cout << "(" << pt.x << ", " << pt.y << "), ";
    }
    cout << endl;

    // union (append)
    res.insert(res.end(), M2.begin(), M2.end());
    cout << "Union of remaining M1 and M2:\n";
    for (size_t i = 0; i < res.size(); i++)
    {
        cout << "(" << res[i].x << ", " << res[i].y << "), ";
    }
    cout << endl;

    return res;
}

```

```

int main() {
    // Karatsuba Example
    cout << "=== Karatsuba Multiplication ===" << endl;
    int x = 1234, y = 5678;
    int result = karatsuba(x, y);
    cout << "Final Result: " << x << " * " << y << " = " << result << endl << endl;
    // Strassen Example
    cout << "=== Strassen's Matrix Multiplication ===" << endl;
    vector<vector<long long>> A = {
        {1, 2, 3},
        {4, 5, 6}
    };

    vector<vector<long long>> B = {
        {7, 8},
        {10, 11},
    };
}

```

```

        {13, 14}
    };

    printMatrix(A, "Matrix A");
    printMatrix(B, "Matrix B");

    auto C = multiplyStrassen(A, B);
    printMatrix(C, "Result C = A * B");

    vector<Point> S = {{1,2},{2,1},{2,3},{3,2},{4,4},{4,1},{5,3},{5,5}};
    cout << "=== Divide and Conquer Maxima Set ===\n";
    // for (size_t i = 0; i < S.size() ; i++)
    // {
    //     cout << "(" << S[i].x << ", " << S[i].y << "), ";
    //     /* code */
    // }
    cout << endl;

    auto M = maximaSet(S);

    cout << "Maxima points:\n";
    for (auto &pt : M) cout << "(" << pt.x << ", " << pt.y << ")\n";

    return 0;
}

```